STRIPE PROJECT

5. Security & Compliance

D6. Security & Compliance Plan

> *Nicolas Pichon - AIA RNCP 38777 / BC02 / D6 : Security & Compliance Plan / v04 - 2026/11/05.*

- D6.1. Contexte
- D6.2. Chiffrement des données
  - D6.2.1. Données _en transit_
  - D6.2.2. Données _au repos_
- D6.3. Contrôle des accès
- D6.4. Journalisation d'audit
- D6.5. Conformité RGPD / CCPA
- D6.6. Conformité PCI-DSS
- D6.7. Détection des incidents de sécurité
- D6.8. Automatisation du reporting et du monitoring
  - D6.8.1. Rapports _a posteriori_ : les tables d'agrégats analytiques
  - D6.8.2. Règles d'alerte en _temps réel_
- Références

D6.1. Contexte

Ce document présente le plan de sécurisation et de mise en conformité réglementaire du système de données *Stripe*, présenté dans le cas d'étude [D0]. Conformément aux exigences du cas d'étude, le plan couvre les problématiques suivantes :

- Sécurisation :
  - chiffrement des données (§D6.2),
  - contrôle des accès (§D6.2),
  - journalisation des opérations à destination des audits (§D6.2)
- Conformité réglementaire :
  - automatisation de la surveillance de la conformité réglementaire
  - automatisation du reporting associés.

> Le système considéré est composé des sous-systèmes de données *OLTP* [D2], *OLAP* [D3], NoSQL [D4], l'architecture d'intégration des trois sous-systèmes étant décrite dans [D5].

D6.2. Chiffrement des données

D6.2.1. Données *en transit*

TLS 1.2 minimum sur toutes les connexions :
- client ↔ *OLTP*,
- *OLTP* ↔ *Kafka*,
- *Kafka* ↔ *NoSQL*/*OLAP*,
- API de reporting ↔ marchand.

Aucune exception, *y compris* dans le trafic interne entre composants du même réseau privé.

D6.2.2. Données *au repos*

Système	Technologie	Mécanisme
OLTP	*PostgreSQL*	Chiffrement du volume de stockage (transparent)
OLAP	*Snowflake*	Chiffrement natif au repos
NoSQL	*MongoDB*	Chiffrement natif au repos
Sauvegardes et exports		Chiffrement systématique

Remarques

OLTP / Chiffrement du volume de stockage	Couvre aussi les sauvegardes *WAL* (cf. [D2-D])
OLAP / Chiffrement natif au repos	Standard chez tous les fournisseurs managés (*Redshift*/*BigQuery*/*Snowflake*)
NoSQL / Chiffrement natif au repos	Couvre les collections et leurs index
Sauvegarde / Chiffrement systématique	Doit couvrir également les exportations de données

Gestion des clés de chiffrement. Service de gestion de clés centralisé (KMS du fournisseur cloud), avec rotation périodique et séparation des responsabilités. Aucune personne seule ne détient l'ensemble des clés de déchiffrement de production.

Non Rétention du PAN. Le numéro de carte bancaire (PAN) n'est jamais stocké en entier (cf. `PaymentMethod.card_last4` [D2-B]). C'est une décision de réduction de périmètre imposée par *PCI-DSS*.

D6.3. Contrôle des accès

Principe : Role-based Access Control (RBAC)

> Les privilèges sont accordés aux *rôles* plutôt qu'aux individus.

Rôles d'accès et acteurs d'audit

Les rôles d'accès sont compatibles avec la typologie des acteurs déjà posée par la classe analytique `AuditActorType` dans [D2-B] :

Rôle	Type d'acteur
Client Terminal	`user`
Agent de Support	`user`
Responsable Conformité	`user`
Service de Scoring de Fraude	`service`
Processus (Batch/ETL)	`system`
API Marchande	`merchant_api`
Administrateur Plateforme	`user`

Règles d'accès

Rôle	Portée d'accès
Client Terminal	Ses propres données uniquement
Agent de Support	Lecture, scoping RLS par marchand assigné
Responsable Conformité	Lecture transverse des tables de conformité (mais pas des données de paiement)
Service de Scoring de Fraude	Lecture `Transaction` /NoSQL features, écriture `FraudScore` uniquement
Processus (Batch/ETL)	Lecture seule, réplicas dédiés
API Marchande	Lecture/écriture strictement limité au `merchant_id`
Administrateur Plateforme	Élevé, exceptionnel

> **Les acteurs doivent-ils se justifier obligatoirement ?**

Rôle	Justification obligatoire?
Client Terminal	on (self-service)
Agent de Support	Oui (sur les actions sensibles)
Responsable Conformité	Oui (systématique)
Service de Scoring de Fraude	Non (automatisé, tracé)
Processus (Batch/ETL)	Non (automatisé, tracé)
API Marchande	Non (*RLS* suffit)
Administrateur Plateforme	Oui (systématique, revue a posteriori)

> Les champs `is_sensitive` & `requires_justification` de la table `AuditAction` [D2-B] opérationnalisent l'obligation de justification des rôles (en particulier pour les *Agents de Support*) : les actions marquées *sensibles* exigent un motif renseigné, vérifié à l'écriture.

Row-Level Security (RLS)

Le contrôle des accès au niveau des lignes est déjà retenue dans le modèle *OLAP* [D3, §D3.5] : un marchand ne peut pas accéder à d'autres enregistrements que ceux que lui accorde les droits d'accès associés à `merchant_id` . Ce niveau de sécurité doit être imposé par le moteur de stockage analytique lui-même.

D6.4. Journalisation d'audit

Le système de journalisation des accès en lecture et en écriture des données transactionnelles est exposé dans [D2-C], pour les tables transactionnelles `DataAccessLog` (consultations) et `ChangeAuditLog` (modifications), et dans [D1;, pour les mécanismes de création des entrées de journaux.

Les points important du système de journalisation sont les suivants :

1. *Les tables de logs sont accessibles en écriture seule (append-only)* --> suppression des droits pour les opérations `UPDATE` / `DELETE`, y compris pour les administrateurs.
2. Les évènements d'écriture (`CREATE` , `UPDATE` , `DELETE`) sur les tables de journalisation transactionnelles (à l'exception de la table d'accès `DataAccessLog`) sont une réplication asynchrone des évènements répertoriés par le mécanisme *CDC* (*Change Data Capture*) du système de stockage transactionnel.
3. La table de journalisation `DataAccessLog` doit être alimentée par *un autre mécanisme asynchrone* car les accès en lecture seule (`SELECT`) ne sont pas détectés par le mécanisme *CDC*.

D6.5. Conformité RGPD / CCPA

Les processus de conformité à superviser sont pris en charge transversalement par les sous-systèmes *OLTP* [D2][D2-B], *OLAP* [D3] [D3-B], *NoSQL* [D4] et par le pipeline d'intégration [D5] :

Consentement

- `ConsentRecord` [D2-B] : accès en écriture seule, un retrait de consentement crée une nouvelle ligne, l'historique des consentements n'est jamais effacé, le consentement est rattaché au marchand mais pas à *Stripe* directement.

Droits de consultation et de modification

- `DataSubjectRequest` [D2-B] porte le délai légal réel en fonction du cadre réglementaire qui s'applique (30 jours RGPD, 45 jours CCPA) ;
- `FactDataSubjectRequest.is_within_deadline` [D3-B] est l'indicateur central du rapport de conformité.

Droit à l'effacement

- [D5, §D5.9.3] développe la stratégie complète du droit à l'effacement mise en place dans le cadre du système de données *Stripe*. Cette stratégie concilie l'effacement exigé par le règlement *RGPD* avec l'immuabilité des journaux d'audit et des topics Kafka non compactés.

Minimisation des données

- *TTL* (*Time To Live*) des documents bruts de `user_sessions` [D4, §D4.5.3] : les données de clickstream ont une durée de vie limitée après leurs agrégations pour la base OLAP.

D6.6. Conformité PCI-DSS

Synthèse des mesures actées ci-dessus :

1. *Ne jamais conserver le PAN dans le périmètre du système Stripe* (§D6.2).
2. *Chiffrer systématiquement les données, au repos et en transit (§D6.2).
3. *Contrôler les accès ligne par ligne sur toutes les données de paiement*
   - `Transaction` , `PaymentMethod` , `Refund` et `Chargeback` entrent dans le périmètre *RLS* comme n'importe quelle donnée sensible (§D6.3).
4. *Journaliser les accès sur toutes les données de paiement*
   - Tous les accès aux tables transactionnelles `Transaction` , `PaymentMethod` , `Refund` et `Chargeback` sont tracés dans `DataAccessLog` (§D6.4).

D6.7. Détection des incidents de sécurité

Le cycle de vie des incidents de sécurité est le suivant (cf. `SecurityIncidentStatus`) :

```
open → investigating → contained → closed
```

Ces états correspondent à un processus de détection et de réponse en cinq étapes :

1. **Détection** → `open` + `detected_at`
   - > alerting automatisé (§D6.8) ou signalement manuel ;
   - un nouvel enregistrement est créé dans `SecurityIncident` ;
   - le champ `detected_at` horodate le déclenchement du délai légal.
2. **Qualification** → `investigating`
   - > évaluation de la sévérité et du périmètre affecté.
3. **Confinement** → `contained`
   - > révocation des accès compromis, correction de la faille.
4. **Notification** → `notified_at`
   - > si des données personnelles sont concernées, le champ `notified_at` doit être renseigné avant l'échéance du délai réglementaire (72 heures pour *RGPD*).
5. **Clôture et retour d'expérience** → `resolved_at` + `closed`
   - > un incident clos *sans notification* alors que des données personnelles étaient en jeu doit être *réouvert*.

D6.8. Automatisation du reporting et du monitoring

D6.8.1. Rapports *a posteriori* : les tables d'agrégats analytiques

Les tables d'agrégats analytiques conçues en [D3, §D3.4] sont réutilisées pour alimenter/générer des rapports périodiques, collectant les évènements de sécurité et de conformité *a posteriori* :

- `AggComplianceDailyMerchant` :
  - date, merchant -> total access count , sensitive access count, etc.
- `AggDataSubjectRequestMonthly` :
  - year, month, merchant, request type ->received count, within-deadline count, etc.
- `AggSecurityIncidentMonthly` :
  - year, month, merchant, severity ->incident count, within-72h count, etc.

D6.8.2. Règles d'alerte en *temps réel*

Proposition pour quelques règles d'alerte déclenchées par les données transactionnelle :

Condition	Action
Plusieurs tentatives d'accès non autorisées venant du même acteur et dans une durée courte	Alerter immédiatement l'équipe de sécurité
Transaction bloquée automatiquement (*)	Notifier en temps réel
Demande de modification non traitée (**)	Alerter l'équipe de conformité
Retard de notification d'un incident de sécurité (***)	Escalade automatique *avant* l'échéance des 72h

> - *(*) `fraud_score` -> `risk_level` -> `blocks_transaction` (`transaction_id)*
> - *(**) cf. `DataSubjectRequest` [D2-B]*
> - *(***) cf. `SecurityIncident` + `notified_at` [D2-B]*

Seuils ou évènements de déclenchement de l'action

Condition	Seuil/Evènement
Plusieurs tentatives d'accès non autorisées (même acteur / durée courte)	Tentatives ≥ 3 en 10 min.
Transaction bloquée automatiquement	Tout événement
Demande de modification non traitée	J-5 avant `due_at`
Retard de notification d'un incident de sécurité	H+48 depuis `detected_at`

Flux d'exécution en temps réel

On a déjà défini un mécanisme de détection des évènements de sécurité en temps réel dans [D5, §D5.8.5] : consommateur *Flink*, abonné aux évènements *Kafka* sur le topic `audit.access` .

Références

Spécifications

- [D0] *Stripe Project Introduction*

Livrables

- [D1] *Comprehensive Data Architecture Diagram*
- [D2] *OLTP Entity-Relation Diagrams*
- [D3] *OLAP System Schema Design*
- [D4] *NoSQL Data Model*
- [D5] *Data Pipeline Architecture*

Annexes

- [D2-B] *OLTP Physical Data Model*
- [D2-C] *OLTP Design Notes*
- [D3-B] *OLAP Physical Data Model*